

Module 16: Exception Handling in Distributed APIs

Learning Goal

By the end of this module, you should be able to design API error handling that is clear, secure, predictable, and useful to client applications.

In a normal Java program, an exception might simply stop execution. In a distributed API, failure is expected. A request can fail because the client sent bad data, another service timed out, the database is unavailable, authentication failed, a business rule was violated, or the code hit an unexpected bug.

Your job as a backend engineer is not to avoid every possible error. Your job is to make failures controlled, understandable, secure, and useful.

Why This Matters

Imagine this API call:

```
POST /accounts/transfer
```

The client sends:

```
{
  "fromAccount": "A100",
  "toAccount": "B200",
  "amount": 5000
}
```

Several things could go wrong:

```
fromAccount does not exist
amount is negative
balance is too low
database is down
payment service times out
user is not authorized
unexpected NullPointerException
```

If your API returns only this:

```
{
  "error": "Something went wrong"
}
```

the client cannot respond intelligently.

A better response is structured and predictable:

```
{
  "timestamp": "2026-08-01T14:22:10Z",
  "status": 400,
  "error": "Bad Request",
  "code": "INVALID_TRANSFER_AMOUNT",
  "message": "Transfer amount must be greater than zero",
  "path": "/accounts/transfer"
}
```

Common API Error Categories

1. Client Errors

The caller did something wrong.

Common HTTP status codes:

```
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
422 Unprocessable Entity
```

Example:

```
throw new InvalidTransferAmountException("Transfer amount must be greater than zero");
```

This should become:

```
400 Bad Request
```

2. Business Rule Errors

The request is structurally valid, but it violates a business rule.

Examples:

```
Insufficient balance
Account is frozen
Cannot transfer to the same account
Duplicate username
```

These commonly return:

```
409 Conflict
```

or sometimes:

```
422 Unprocessable Entity
```

3. Dependency Errors

Your API depends on another system, and that system failed.

Examples:

```
Database unavailable
Inventory service timeout
Payment gateway rejected request
Message broker disconnected
```

These often return:

```
502 Bad Gateway
503 Service Unavailable
504 Gateway Timeout
```

4. Server Bugs

Something unexpected happened in your code.

Examples:

```
NullPointerException
IllegalStateException
ArrayIndexOutOfBoundsException
```

These usually return:

```
500 Internal Server Error
```

Important rule: never expose raw stack traces to the client.

Bad:

```
{
  "error": "java.lang.NullPointerException at com.bank.TransferService..."
}
```

Good:

```
{
  "status": 500,
  "code": "INTERNAL_ERROR",
  "message": "An unexpected error occurred"
}
```

Standard Error Response Model

A good API should return errors in a consistent shape.

```
public class ApiError {
    private String timestamp;
    private int status;
    private String error;
    private String code;
    private String message;
    private String path;

    public ApiError(String timestamp, int status, String error, String code, String message,
String path) {
        this.timestamp = timestamp;
        this.status = status;
        this.error = error;
        this.code = code;
        this.message = message;
        this.path = path;
    }

    // getters
}
```

A client should not have to guess whether the error message is in `error` , `msg` , `description` , or `details` .

Consistency is mercy.

Spring Boot Global Exception Handling

In Spring Boot, a common pattern is to centralize error handling using `@ControllerAdvice` or `@RestControllerAdvice` .

Instead of handling exceptions separately in every controller, define one global handler.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(AccountNotFoundException.class)
    public ResponseEntity<ApiError> handleAccountNotFound(
        AccountNotFoundException ex,
        HttpServletRequest request
    ) {
        ApiError error = new ApiError(
            Instant.now().toString(),
            404,
            "Not Found",
            "ACCOUNT_NOT_FOUND",
            ex.getMessage(),
            request.getRequestURI()
        );
    }
}
```

```

        );

        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }
}

```

Now if any controller throws `AccountNotFoundException`, Spring returns a clean `404` response.

Custom Exceptions

Create custom exceptions for meaningful domain failures.

```

public class AccountNotFoundException extends RuntimeException {
    public AccountNotFoundException(String accountId) {
        super("Account not found: " + accountId);
    }
}

```

Service code:

```

public Account getAccount(String accountId) {
    return accountRepository.findById(accountId)
        .orElseThrow(() -> new AccountNotFoundException(accountId));
}

```

The service focuses on business logic. The exception handler focuses on HTTP response formatting.

Fault Propagation

Fault propagation means deciding how errors move through the system.

Example:

```

Client -> Order API -> Payment API -> Bank API

```

If the Bank API times out, the Order API should not return this:

```

{
  "message": "SocketTimeoutException in BankHttpClient.java line 84"
}

```

It should return something safe and useful:

```

{
  "status": 503,
  "code": "PAYMENT_SERVICE_UNAVAILABLE",
  "message": "Payment processing is temporarily unavailable"
}

```

Internally, log the technical details. Externally, return a client-safe message.

Secure Error Reporting

Never expose:

```
stack traces
database table names
SQL queries
internal hostnames
secret keys
file paths
class names
implementation details
```

Bad:

```
{
  "message": "SQL error on table user_credentials: password_hash column cannot be null"
}
```

Good:

```
{
  "code": "USER_CREATION_FAILED",
  "message": "Unable to create user account"
}
```

The client gets enough to act. Attackers do not get a map of your system.

Mental Model

Think of exception handling in distributed APIs as three layers:

```
Exception
  What went wrong inside Java?

HTTP status
  What should the client understand?

Error body
  What useful, safe message should be returned?
```

Examples:

```
AccountNotFoundException
-> 404 Not Found
-> ACCOUNT_NOT_FOUND
```

```
InsufficientFundsException  
-> 409 Conflict  
-> INSUFFICIENT_FUNDS
```

```
PaymentServiceTimeoutException  
-> 504 Gateway Timeout  
-> PAYMENT_TIMEOUT
```

```
Unexpected NullPointerException  
-> 500 Internal Server Error  
-> INTERNAL_ERROR
```

Practice Exercises

Exercise 1: Build a Standard Error Model

Create a class called `ApiError` with fields like:

```
timestamp  
status  
error  
code  
message  
path
```

Then manually return it from a controller when something fails.

Goal: learn what a clean API error response should look like.

Exercise 2: Create Custom Exceptions

Create these custom runtime exceptions:

```
AccountNotFoundException  
InvalidTransferAmountException  
InsufficientFundsException  
PaymentServiceUnavailableException
```

Use them inside a simple `TransferService`.

Goal: separate business failure names from generic Java exceptions.

Exercise 3: Add a Global Exception Handler

Create a Spring Boot `@RestControllerAdvice` class and map exceptions:

```
InvalidTransferAmountException -> 400 Bad Request  
AccountNotFoundException -> 404 Not Found  
InsufficientFundsException -> 409 Conflict
```

```
PaymentServiceUnavailableException -> 503 Service Unavailable  
Exception -> 500 Internal Server Error
```

Goal: stop writing error handling inside every controller.

Exercise 4: Validation Error Handling

Create a request DTO:

```
public class TransferRequest {  
    @NotBlank  
    private String fromAccount;  
  
    @NotBlank  
    private String toAccount;  
  
    @Positive  
    private BigDecimal amount;  
}
```

Then handle validation failures and return a useful error response.

Goal: distinguish bad input from business rule failure.

Exercise 5: Safe Error Messages

Intentionally throw a `NullPointerException` or database-like error, then make sure the client only sees:

```
{  
  "code": "INTERNAL_ERROR",  
  "message": "An unexpected error occurred"  
}
```

Your server log should contain the real exception.

Goal: protect internal system details.

Exercise 6: Simulate a Downstream Service Timeout

Create a fake `PaymentClient` method:

```
public void charge() {  
    throw new PaymentServiceUnavailableException("Payment provider timed out");  
}
```

Return a client-safe response:

```
503 Service Unavailable
```

Goal: practice handling dependency failure.

Exercise 7: Map Errors to Correct HTTP Status Codes

Given these cases, decide the status code:

```
Negative transfer amount
Missing account ID
Account not found
Duplicate username
Insufficient funds
Payment provider timeout
Unexpected server bug
Unauthorized user
Authenticated user lacks permission
```

Expected direction:

```
400
400
404
409
409
504 or 503
500
401
403
```

Exercise 8: Add Correlation IDs

Add a field to `ApiError` :

```
requestId
```

Generate one per request or read it from a header like:

```
X-Request-ID
```

Goal: learn how distributed systems trace errors across services.

Exercise 9: Write Tests for Exception Responses

Using JUnit and MockMvc, test that:

```
invalid amount returns 400
missing account returns 404
insufficient funds returns 409
unexpected exception returns 500
```

Goal: prove your API error contract works.

Exercise 10: Mini Project

Build a small API:

```
POST /transfers
GET /accounts/{id}
POST /users
```

Include:

```
custom exceptions
global exception handler
standard error response
validation handling
safe 500 response
logs for internal errors
```

Target deliverable: a Spring Boot API where every failure returns a predictable JSON error body and the correct HTTP status code.

Lab: Exception Handling in Distributed APIs

Lab Goal

Build a small Spring Boot banking API that handles failures cleanly using:

```
custom exceptions
@RestControllerAdvice
standard JSON error responses
proper HTTP status codes
safe internal error handling
```

Scenario

You are building a transfer API:

```
POST /api/transfers
```

The API should allow money transfers between accounts, but it must handle common API failures properly.

Step 1: Create the Error Response Model

Create `ApiError.java` :

```
package com.example.demo.error;

import java.time.Instant;

public class ApiError {
    private String timestamp;
```

```

private int status;
private String error;
private String code;
private String message;
private String path;

public ApiError(int status, String error, String code, String message, String path) {
    this.timestamp = Instant.now().toString();
    this.status = status;
    this.error = error;
    this.code = code;
    this.message = message;
    this.path = path;
}

public String getTimestamp() { return timestamp; }
public int getStatus() { return status; }
public String getError() { return error; }
public String getCode() { return code; }
public String getMessage() { return message; }
public String getPath() { return path; }
}

```

Step 2: Create Custom Exceptions

```

package com.example.demo.error;

public class AccountNotFoundException extends RuntimeException {
    public AccountNotFoundException(String accountId) {
        super("Account not found: " + accountId);
    }
}

```

```

package com.example.demo.error;

public class InvalidTransferAmountException extends RuntimeException {
    public InvalidTransferAmountException() {
        super("Transfer amount must be greater than zero");
    }
}

```

```

package com.example.demo.error;

public class InsufficientFundsException extends RuntimeException {
    public InsufficientFundsException() {
        super("Insufficient funds for this transfer");
    }
}

```

Step 3: Create Request DTO

```
package com.example.demo.transfer;

import java.math.BigDecimal;

public class TransferRequest {
    private String fromAccount;
    private String toAccount;
    private BigDecimal amount;

    public String getFromAccount() { return fromAccount; }
    public void setFromAccount(String fromAccount) { this.fromAccount = fromAccount; }

    public String getToAccount() { return toAccount; }
    public void setToAccount(String toAccount) { this.toAccount = toAccount; }

    public BigDecimal getAmount() { return amount; }
    public void setAmount(BigDecimal amount) { this.amount = amount; }
}
```

Step 4: Create Service Logic

```
package com.example.demo.transfer;

import com.example.demo.error.AccountNotFoundException;
import com.example.demo.error.InsufficientFundsException;
import com.example.demo.error.InvalidTransferAmountException;
import org.springframework.stereotype.Service;

import java.math.BigDecimal;
import java.util.Map;

@Service
public class TransferService {

    private final Map<String, BigDecimal> accounts = Map.of(
        "A100", new BigDecimal("1000.00"),
        "B200", new BigDecimal("500.00")
    );

    public String transfer(TransferRequest request) {
        if (request.getAmount() == null || request.getAmount().compareTo(BigDecimal.ZERO) <= 0) {
            throw new InvalidTransferAmountException();
        }

        if (!accounts.containsKey(request.getFromAccount())) {
            throw new AccountNotFoundException(request.getFromAccount());
        }
    }
}
```

```

        if (!accounts.containsKey(request.getToAccount())) {
            throw new AccountNotFoundException(request.getToAccount());
        }

        BigDecimal balance = accounts.get(request.getFromAccount());

        if (balance.compareTo(request.getAmount()) < 0) {
            throw new InsufficientFundsException();
        }

        return "Transfer completed successfully";
    }
}

```

Step 5: Create Controller

```

package com.example.demo.transfer;

import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/transfers")
public class TransferController {

    private final TransferService transferService;

    public TransferController(TransferService transferService) {
        this.transferService = transferService;
    }

    @PostMapping
    public String transfer(@RequestBody TransferRequest request) {
        return transferService.transfer(request);
    }
}

```

Step 6: Create Global Exception Handler

```

package com.example.demo.error;

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

@RestControllerAdvice
public class GlobalExceptionHandler {

```

```

@ExceptionHandler(InvalidTransferAmountException.class)
public ResponseEntity<ApiError> handleInvalidAmount(
    InvalidTransferAmountException ex,
    HttpServletRequest request
) {
    ApiError error = new ApiError(
        400,
        "Bad Request",
        "INVALID_TRANSFER_AMOUNT",
        ex.getMessage(),
        request.getRequestURI()
    );

    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
}

@ExceptionHandler(AccountNotFoundException.class)
public ResponseEntity<ApiError> handleAccountNotFound(
    AccountNotFoundException ex,
    HttpServletRequest request
) {
    ApiError error = new ApiError(
        404,
        "Not Found",
        "ACCOUNT_NOT_FOUND",
        ex.getMessage(),
        request.getRequestURI()
    );

    return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
}

@ExceptionHandler(InsufficientFundsException.class)
public ResponseEntity<ApiError> handleInsufficientFunds(
    InsufficientFundsException ex,
    HttpServletRequest request
) {
    ApiError error = new ApiError(
        409,
        "Conflict",
        "INSUFFICIENT_FUNDS",
        ex.getMessage(),
        request.getRequestURI()
    );

    return ResponseEntity.status(HttpStatus.CONFLICT).body(error);
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ApiError> handleGenericException(
    Exception ex,

```

```
        HttpServletRequest request
    ) {
        ApiError error = new ApiError(
            500,
            "Internal Server Error",
            "INTERNAL_ERROR",
            "An unexpected error occurred",
            request.getRequestURI()
        );

        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
    }
}
```

Step 7: Test with Postman or curl

Valid request:

```
{
  "fromAccount": "A100",
  "toAccount": "B200",
  "amount": 100
}
```

Expected:

```
Transfer completed successfully
```

Invalid amount:

```
{
  "fromAccount": "A100",
  "toAccount": "B200",
  "amount": -50
}
```

Expected status:

```
400 Bad Request
```

Missing account:

```
{
  "fromAccount": "X999",
  "toAccount": "B200",
  "amount": 100
}
```

Expected status:

404 Not Found

Too much money:

```
{
  "fromAccount": "A100",
  "toAccount": "B200",
  "amount": 5000
}
```

Expected status:

409 Conflict

Challenge Task

Add one more exception:

PaymentServiceUnavailableException

Then simulate a downstream service failure and return:

503 Service Unavailable

with this error code:

PAYMENT_SERVICE_UNAVAILABLE

Lab Success Criteria

You are done when:

- All errors return JSON
- Each error has the correct HTTP status
- Business exceptions are handled separately
- Unexpected errors return a safe message
- No stack trace is exposed to the client